

Tugas Besar 1 IF3270 Pembelajaran Mesin

Feedforward Neural Network



Kelompok 28 MagangLearning

Nicholas Andhika Lucas	13523014
Stefan Matthew Susanto	13523020
Kenneth Ricardo Chandra	13523022

BANDUNG

INSTITUT TEKNOLOGI BANDUNG

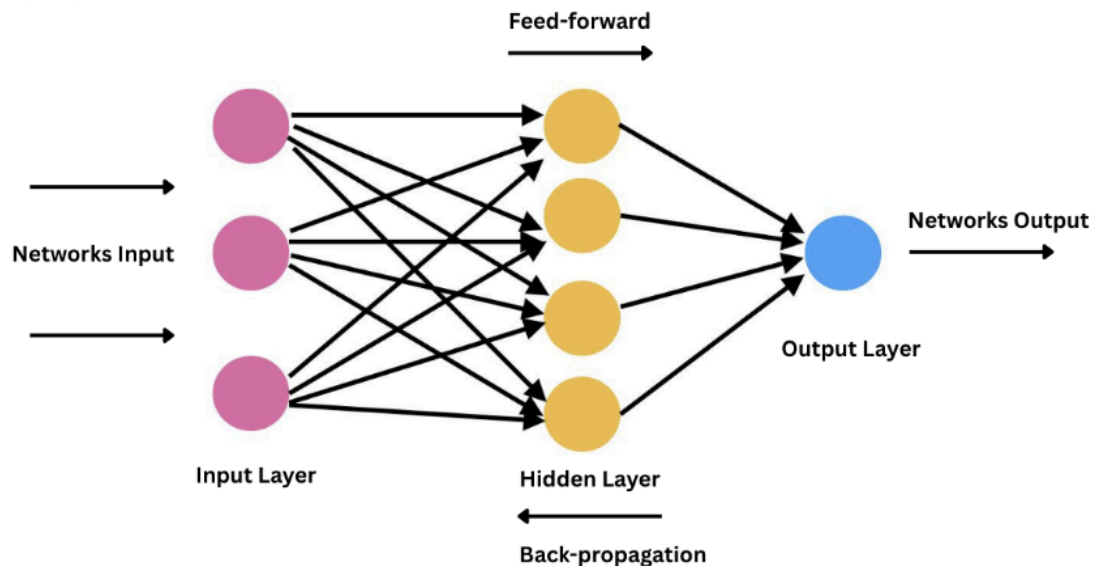
2026

DAFTAR ISI

DAFTAR ISI.....	2
1. Deskripsi Persoalan.....	3
2. Pembahasan.....	3
2.1. Penjelasan Implementasi.....	3
2.1.1. Deskripsi kelas beserta deskripsi atribut dan methodnya.....	3
2.1.2. Penjelasan forward propagation.....	16
2.1.3. Penjelasan backward propagation dan weight update.....	17
2.1.4. Penjelasan Implementasi Bonus.....	19
2.2. Hasil Pengujian.....	19
2.2.1. Pengaruh depth dan width.....	21
2.2.2. Pengaruh fungsi aktivasi.....	23
2.2.3. Pengaruh learning rate.....	25
2.2.4. Pengaruh inisialisasi bobot.....	27
2.2.5. Pengaruh regularisasi.....	28
2.2.6. Perbandingan dengan library sklearn.....	30
2.2.7. Perbandingan dengan RMS Normalization.....	30
3. Kesimpulan dan Saran.....	32
3.1. Kesimpulan.....	32
3.2. Saran.....	32
4. Pembagian Tugas.....	33
5. Referensi.....	33
6. Lampiran Form Penggunaan AI.....	35

1. Deskripsi Persoalan

Tugas Besar ini bertujuan untuk mengimplementasikan model Feedforward Neural Network (FFNN) *from scratch* menggunakan bahasa Python. FFNN merupakan salah satu dasar dari Artificial Neural Network (ANN) dalam subjek *machine learning*, di mana informasi yang masuk dan diproses dalam jaringan ini hanya bergerak maju dari input layer, hidden layer, dan output layer. Tugas ini akan memodelkan kelas-kelas pembentuk FFNN seperti fungsi aktivasi, fungsi loss, dan layer, serta algoritma-algoritma yang esensial dalam cara kerja FFNN berupa *back propagation*, dengan tujuan menentukan pengaruh dari pengaturan parameter terhadap hasil prediksi dan model FFNN.



2. Pembahasan

2.1. Penjelasan Implementasi

2.1.1. Deskripsi kelas beserta deskripsi atribut dan methodnya

1. Class Activation

Kelas ini berfungsi untuk melakukan perhitungan fungsi aktivasi dari tiap layer. Kelas ini terdiri dari berbagai fungsi aktivasi, mencakup fungsi Linear, ReLU, Sigmoid, Hyperbolic Tangent, serta bonus fungsi aktivasi lain yaitu Softmax, Binary Step, dan Leaky ReLU yang akan mengembalikan output

dari tiap fungsi, termasuk opsi untuk menghitung menggunakan fungsi turunan pertamanya.

```
class Activation:
    @staticmethod
    def linear(x, derivative=False):
        if derivative:
            return 1
        return x

    @staticmethod
    def relu(x, derivative=False):
        if derivative:
            return np.where(x > 0, 1, 0)
        return np.maximum(0, x)

    @staticmethod
    def sigmoid(x, derivative=False):
        sig = 1 / (1 + np.exp(-x))
        if derivative:
            return sig * (1 - sig)
        return sig

    @staticmethod
    def tanh(x, derivative=False):
        tanh = np.tanh(x)
        if derivative:
            return 1 - tanh ** 2
        return tanh

    @staticmethod
    def softmax(x, derivative=False):
        exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
        softmax = exp_x / np.sum(exp_x, axis=1, keepdims=True)
        if derivative:
            return softmax * (1 - softmax)
        return softmax

    @staticmethod
    def binary_step(x, derivative=False):
        if derivative:
            return 0
        return np.where(x >= 0, 1, 0)

    @staticmethod
    def softsign(x, derivative=False):
        if derivative:
            return 1 / (1 + np.abs(x)) ** 2
        return x / (1 + np.abs(x))

    @staticmethod
    def leaky_relu(x, derivative=False):
        if derivative:
            return np.where(x > 0, 1, 0.01)
        return np.where(x > 0, x, 0.01 * x)
```

2. Class Loss

Kelas ini berfungsi untuk menghitung fungsi loss dari model FFNN yang terdiri dari metode Mean Squared Error (MSE), Binary Cross Entropy (BCE), dan Categorical Cross Entropy (CCE), dengan opsi perhitungan menggunakan fungsi turunan pertama.

```
class Loss:
    @staticmethod
    def mse(y_true, y_pred, derivative=False):
        # y_true, y_pred = arr
        if derivative:
            return 2 * (y_pred - y_true) / y_true.shape[0]
        return np.mean((y_true - y_pred) ** 2)

    @staticmethod
    def binary_cross_entropy(y_true, y_pred, derivative=False):
        # y_true, y_pred = arr
        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
        if derivative:
            return (y_pred - y_true) / (y_pred * (1 - y_pred)) / y_true.shape[0]
        return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

    @staticmethod
    def categorical_cross_entropy(y_true, y_pred, derivative=False):
        # y_true, y_pred = arr
        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
        if derivative:
            return -y_true / y_pred / y_true.shape[0]
        return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))
```

3. Class Layer

Kelas ini berfungsi sebagai pemodelan lapisan layer dalam neural network yang dibangun. Atribut utama dari kelas layer adalah input size, output size, fungsi aktivasi dan metode inisialisasi bobot pilihan. Terdapat atribut lain yang merupakan variabel dari salah satu metode inisialisasi bobot, seperti lower dan upper (untuk metode uniform), serta mean dan var (untuk metode random normal). Terdapat juga atribut seed yang dapat digunakan untuk mengatur *randomness* dan *reproducibility* dari inisialisasi bobot.

Fungsi yang terdapat pada kelas ini adalah initialize weight untuk menginisialisasi bobot menggunakan metode pilihan, yaitu zero, uniform, random normal, dan tambahan bonus berupa xavier dan he. Untuk inisialisasi bobot, RNG dimodelkan secara manual untuk memungkinkan pengaturan *reproducibility* menggunakan parameter seed.

```
class Layer:
    def __init__(self, input_size, output_size, activation, init_method='random_normal',
```

```

seed=None, lower=-0.5, upper=0.5, mean=0, var=1, norm=None ):
    self.input_size = input_size
    self.output_size = output_size
    self.init_method = init_method
    self.seed = seed
    self.lower = lower
    self.upper = upper
    self.mean = mean
    self.var = var
    self.norm = norm

    self.weights = None
    self.initialize_weights()

    self.bias = np.zeros((1, output_size))
    self.activation_name = activation
    self.input = None
    self.z = None

    self.dweights = np.zeros_like(self.weights)
    self.dbias = np.zeros_like(self.bias)
    if self.norm == 'rmsnorm':
        self.rmsnorm = RMSNorm(output_size)
    if self.seed is not None:
        self.initialize_weights()

def initialize_weights(self, rng=None):
    if self.seed is not None:
        rng = np.random.default_rng(self.seed)
    elif rng is None:
        rng = np.random.default_rng()

    if self.init_method == 'zero':
        self.weights = np.zeros((self.input_size, self.output_size))

    elif self.init_method == 'uniform':
        self.weights = rng.uniform(self.lower, self.upper, (self.input_size,
self.output_size))

    elif self.init_method == 'random_normal':
        sd = np.sqrt(self.var)
        self.weights = rng.normal(self.mean, sd, (self.input_size,
self.output_size))

    elif self.init_method == 'xavier':
        limit = np.sqrt(6 / (self.input_size + self.output_size))
        self.weights = rng.uniform(-limit, limit, (self.input_size,
self.output_size))

    elif self.init_method == 'he':
        std = np.sqrt(2 / self.input_size)

```

```

        self.weights = rng.normal(0, std, (self.input_size, self.output_size))

    else: # default random_normal
        self.weights = rng.uniform(self.lower, self.upper, (self.input_size,
self.output_size))

    self.dweights = np.zeros_like(self.weights)

```

4. Class FFNN

Kelas ini merupakan kelas pemodelan utama dari Feedforward Neural network. Sesuai dengan model arsitekturnya, atribut dari kelas ini terdiri dari daftar objek Layer, jenis fungsi loss yang dipilih, serta masukan seed dan rng untuk mengatur *reproducibility* dalam pelatihan.

Metode untuk inisialisasi model FFNN adalah `add_layer`, untuk menambahkan layer-layer ke dalam model FFNN. Kemudian, metode utama sesuai dengan algoritmanya adalah `forward` dan `backward` sebagai pemodelan forward pass dan backward pass. Forward pass akan melakukan perhitungan dari input dengan output yang melalui fungsi aktivasi. Sedangkan, backward pass akan melakukan perhitungan loss pada bias dan weight. Setelah melakukan backward pass, fungsi `update_weights` akan melakukan update bobot tersebut. Algoritma dari FFNN ini tercakup dalam fungsi `fit` yang akan melakukan pelatihan model FFNN berdasarkan input data dan parameter masukan, yang akan menyimpan data dari loss untuk setiap epoch.

Terdapat metode helper lain seperti `load` dan `save` untuk menyimpan dan memasukkan model FFNN, `show_gradient` dan `show_weights` untuk menampilkan data pada terminal.

```

class FFNN:
    def __init__(self, loss_function='binary_cross_entropy', seed=None):
        self.layers = []
        self.loss_function = loss_function
        self.seed = seed
        self.rng = np.random.default_rng(seed) if seed is not None else
np.random.default_rng()

    def add_layer(self, layer):
        if layer.seed is None:
            layer.initialize_weights(rng=self.rng)
        self.layers.append(layer)

```

```

def forward(self, x_batch):
    current_input = x_batch

    for layer in self.layers:
        layer.input = current_input
        layer.z = np.dot(current_input, layer.weights) + layer.bias

        if hasattr(layer, 'norm') and layer.norm == 'rmsnorm':
            layer.z = layer.rmsnorm.forward(layer.z)

        activation_func = getattr(Activation, layer.activation_name)
        current_input = activation_func(layer.z)
    return current_input

def compute_loss(self, y_true, y_pred, derivative=False):
    loss_func = getattr(Loss, self.loss_function)
    return loss_func(y_true, y_pred, derivative)

def backward(self, y_true, y_pred, l1_lambda=0.0, l2_lambda=0.0):
    delta = self.compute_loss(y_true, y_pred, derivative=True)

    for i in reversed(range(len(self.layers))):
        layer = self.layers[i]
        activation_func = getattr(Activation, layer.activation_name)
        d_activation = activation_func(layer.z, derivative=True)

        delta = delta * d_activation

        if hasattr(layer, 'norm') and layer.norm == 'rmsnorm':
            delta = layer.rmsnorm.backward(delta)

        layer.dweights = np.dot(layer.input.T, delta)
        layer.dbias = np.sum(delta, axis=0, keepdims=True)

        if l1_lambda > 0:
            layer.dweights += l1_lambda * np.sign(layer.weights)
        if l2_lambda > 0:
            layer.dweights += l2_lambda * layer.weights

        if i > 0:
            delta = np.dot(delta, layer.weights.T)

def show_weights(self):
    for i, layer in enumerate(self.layers):
        print(f"Layer {i+1} Weights:\n {layer.weights}\n Bias:\n{layer.bias}\n")
    #fungsi nampilin distribusi bobot

def update_weights(self, learning_rate):
    for layer in self.layers:
        layer.weights -= learning_rate * layer.dweights
        layer.bias -= learning_rate * layer.dbias

```



```

        if hasattr(layer, 'norm') and layer.norm == 'rmsnorm':
            layer.rmsnorm.update_weights(Learning_rate)

def show_gradient(self):
    pass

def save(self, filename):
    with open(filename, 'wb') as f:
        pickle.dump(self, f)

    print (f"Model saved to {filename}")

def load(self, filename):
    with open(filename, 'rb') as f:
        self.layers = pickle.load(f)

    print(f"Model loaded from {filename}:")

def fit(self, X_train, y_train, X_val=None, y_val=None, batch_size=32,
Learning_rate=0.01, epochs=100, L1_Lambda=0.0, L2_Lambda=0.0, verbose= 1):
    history = {
        'train_loss': [],
        'val_loss': []
    }

    n= X_train.shape[0]

    for epoch in range(epochs):
        # shuffle data
        indices = np.arange(n)
        self.rng.shuffle(indices)

        X_train = X_train[indices]
        y_train = y_train[indices]

        epoch_loss = 0

        for i in range(0, n, batch_size):
            X_batch = X_train[i:i+batch_size]
            y_batch = y_train[i:i+batch_size]

            y_pred = self.forward(X_batch)

            batch_loss = self.compute_loss(y_batch, y_pred)
            epoch_loss += batch_loss * X_batch.shape[0]

            self.backward(y_batch, y_pred, L1_Lambda, L2_Lambda)

            self.update_weights(Learning_rate)

        avg_train_loss = epoch_loss / n

```

```

        history['train_loss'].append(avg_train_loss)

    # Count validation loss
    if X_val is not None and y_val is not None:
        y_val_pred = self.forward(X_val)
        val_loss = self.compute_loss(y_val, y_val_pred)
        history['val_loss'].append(val_loss)

    if verbose == 1 and ((epoch + 1) % 10 == 0 or epoch == 0):
        status = f"Epoch {epoch + 1}/{epochs}. Train Loss: {avg_train_loss:.4f}"

        if X_val is not None and y_val is not None:
            status += f", Val Loss: {val_loss:.4f}"

        print(status)

    return history

```

5. Class RMSNorm

Kelas ini merupakan kelas normalisasi RMS atau Root Mean Square yang memiliki fungsi-fungsi yang sama dengan di kelas FFNN namun telah diubah agar memiliki nilai yang telah diseragamkan melalui root mean square.

```

class RMSNorm:
    def __init__(self, size, eps=1e-8):
        self.gamma = np.ones((1, size))
        self.beta = np.zeros((1, size))
        self.eps = eps

        self.x = None
        self.x_norm = None
        self.rms = None

        self.dgamma = np.zeros_like(self.gamma)
        self.dbeta = np.zeros_like(self.beta)

    def forward(self, x):
        self.x = x
        self.rms = np.sqrt(np.mean(x**2, axis=-1, keepdims=True) + self.eps)
        self.x_norm = x / self.rms
        return self.gamma * self.x_norm + self.beta

    def backward(self, dout):
        _, D = dout.shape

        self.dgamma = np.sum(dout * self.x_norm, axis=0, keepdims=True)
        self.dbeta = np.sum(dout, axis=0, keepdims=True)

```

```

dx_norm = dout * self.gamma
drms = np.sum(dx_norm * self.x * (-1.0 / (self.rms**2)), axis=-1,
keepdims=True)
dx = (dx_norm / self.rms) + (drms * self.x / (D * self.rms))
return dx

def update_weights(self, Learning_rate):
    self.gamma -= Learning_rate * self.dgamma
    self.beta -= Learning_rate * self.dbeta

```

2.1.2. Penjelasan forward propagation

Forward Propagation adalah proses menghitung output dari neural network berdasarkan data yang melewati input layer hingga sampai output layer. Forward propagation akan bergerak maju dan menghitung output setiap lapisan/layer berdasarkan bobot, bias dan fungsi aktivasi. Data mentah yang telah melewati tahap preprocessing akan masuk dan menjadi input layer. Input yang telah diproses akan melewati satu atau lebih hidden layer. Dalam proses itu dilakukan perhitungan nilai output sebagai berikut:

$$z = w * X + b$$

dengan z sebagai output, w sebagai bobot dari tiap input, X sebagai nilai vektor input, dan b adalah layer bias.

Setelah itu, fungsi aktivasi dilakukan berdasarkan nilai output tersebut untuk menerapkan non-linearitas pada batasan keputusan yang relatif kompleks. Fungsi aktivasi dapat menggunakan linear, ReLU, Sigmoid, Hyperbolic Tangent (tanh), ataupun softmax.

Nama Fungsi Aktivasi	Definisi Fungsi
Linear	$Linear(x) = x$
ReLU	$ReLU(x) = \max(0, x)$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent (tanh)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$

Proses perhitungan akan dilakukan berulang dengan output dari tiap layer akan menjadi masukan untuk layer selanjutnya hingga layer output. Hasil dari output layer akan menjadi prediksi data dan dapat dilanjutkan untuk menghitung loss pada tahap Backward propagation.

2.1.3. Penjelasan backward propagation dan weight update

Backward propagation merupakan metode untuk menghitung gradien loss terhadap semua bobot dan bias yang lalu akan digunakan untuk melakukan weight update.

Backward propagation seperti namanya berjalan mundur dari output dengan menggunakan turunan dari fungsi aktivasi yang digunakan pada setiap layer untuk mendapatkan gradien tiap bobot terhadap loss function. Perhitungan dilakukan dengan menggunakan konsep chain rule

Nama Fungsi Aktivasi	Turunan Pertama
----------------------	-----------------

Linear	$\frac{d(\text{Linear}(x))}{dx} = 1$
ReLU	$\frac{d(\text{ReLU}(x))}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$
Sigmoid	$\frac{d(\sigma(x))}{dx} = \sigma(x)(1 - \sigma(x))$
Hyperbolic Tangent (tanh)	$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^x - e^{-x}} \right)^2$
Softmax	Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $\frac{d(\text{softmax}(\vec{x}))}{d\vec{x}} = \begin{bmatrix} \frac{\partial(\text{softmax}(\vec{x}))_1}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x}))_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(\text{softmax}(\vec{x}))_n}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x}))_n}{\partial x_n} \end{bmatrix}$ Dimana untuk $i, j \in \{1, \dots, n\}$, $\frac{\partial(\text{softmax}(\vec{x}))_i}{\partial x_j} = \text{softmax}(\vec{x})_i (\delta_{i,j} - \text{softmax}(\vec{x})_j)$ $\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$

Nama Fungsi Loss	Definisi Fungsi
MSE	$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \frac{\partial \hat{y}_i}{\partial W}$
Binary Cross-Entropy	$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \frac{\partial \hat{y}_i}{\partial W}$
Categorical Cross-Entropy	$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{y_{ij}}{\hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W}$

Dari fungsi backward akan menghasilkan turunan dari weights dan bias yaitu dweights dan dbias yang akan digunakan untuk weight update.

$$W_{new} = W_{old} - \alpha \left(\frac{\partial \mathcal{L}}{\partial W_{old}} \right)$$

α = Learning rate

Weight update digunakan untuk mendapatkan bobot yang baru dan diimplementasikan menggunakan gradient descent dengan learning rate yang telah ditetapkan. Weight update dilakukan agar hasil prediksi semakin akurat

2.1.4. Penjelasan Implementasi Bonus

Bonus yang dibuat berupa He dan Xavier initialization, fungsi aktivasi, dan juga RMS Normalization.

- Xavier initialization merupakan teknik inisialisasi bobot yang dibuat untuk bekerja optimal pada fungsi aktivasi yang simetris dan berpusat pada nol seperti sigmoid dan tanh.

Xavier menginisialisasi dengan mengambil nilai acak dari distribusi seragam dengan batasan interval berdasarkan input dan output size. Rumus intervalnya berupa

$$\text{limit} = \sqrt{\frac{6}{\text{input_size} + \text{output_size}}}$$

- He initialization merupakan teknik inisialisasi yang dapat menjadi solusi atas kekurangan Xavier, yaitu untuk model fungsi aktivasi asimetris seperti ReLu

He menarik bobot dengan rata-rata 0 dan standar deviasi yang digunakan hanya bergantung pada input size dengan rumus

$$\text{std} = \sqrt{\frac{2}{n_{in}}}$$

- Fungsi aktivasi tambahan yang dimodelkan adalah binary step, softsign, dan leaky relu. Penjelasan fungsi aktivasinya adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi	Turunan Pertama
Binary Step	$\begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	0
Softsign	$\frac{x}{1 + x }$	$\frac{1}{(1 + x)^2}$
Leaky ReLU	$\begin{cases} 0.01x & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$	$\begin{cases} 0.01 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$

- RMSNormalization merupakan teknik normalisasi yang menggunakan root mean square (RMS) dengan tujuan menormalisasi aktivasi neuron ke rata-rata 0 dan membaginya dengan variansi.

Perhitungannya berupa input yang dibagi oleh RMSnya, yang lalu akan diskalakan menggunakan parameter pembobot learnable (parameter gamma)

2.2. Hasil Pengujian

Pengujian dilakukan menggunakan inisialisasi base model dan pelatihan dengan menggunakan parameter sebagai berikut:

```
# *** BASE MODEL ***
# *** 3 hidden layer dengan 8 neuron ***
ffnn_base_model = FFNN(loss_function='binary_cross_entropy', seed=RANDOM_STATE)
ffnn_base_model.add_layer(Layer(input_size=input_dim, output_size=8, activation='relu',
init_method='uniform'))
ffnn_base_model.add_layer(Layer(input_size=8, output_size=8, activation='relu',
init_method='uniform'))
ffnn_base_model.add_layer(Layer(input_size=8, output_size=8, activation='relu',
init_method='uniform'))
ffnn_base_model.add_layer(Layer(input_size=8, output_size=1, activation='sigmoid',
init_method='uniform'))

history_base, f1_base = train_and_evaluate_ffnn(
    model=ffnn_base_model,
    X_train=X_train_np, y_train=y_train,
    X_val=X_val_np, y_val=y_val,
    model_name="FFNN Base Model",
    epochs=100, learning_rate=0.1, batch_size=32, verbose=0)
```

)

Penting untuk diketahui bahwa pengujian menggunakan setup `seed=RANDOM_STATE`, yaitu 42. Seed yang sama digunakan saat menginisialisasi model lain pada saat pengujian dengan tujuan *reproducibility*, agar model terlepas dari faktor *randomness* ketika menganalisis pengaruh parameter tertentu terhadap hasil prediksi model FFNN.

2.2.1. Pengaruh depth dan width

- Base model menggunakan **3 hidden layer** dengan **8 neuron**
- 3 variasi kombinasi width (depth tetap): 2, 4, 5 hidden layer
- 3 variasi depth (width semua layer tetap): 16, 24, 32 neuron

Prediksi terbaik dimiliki oleh model dengan 5 hidden layer dan 4 neuron. Dalam dataset ini, ditemukan bahwa model yang semakin mendalam akan memiliki nilai yang lebih baik. Sedangkan, semakin ramping model tersebut, maka akan semakin baik. Khususnya pada grafik loss, model dengan width yang semakin besar seharusnya memiliki nilai loss terkecil, namun ketika diuji coba dengan prediksi validasi, justru hasil lossnya lebih berfluktuasi dan tidak stabil.

Hasil Akurasi Prediksi

```
=====
FFNN Base Model (3 Hidden Layer) - Data Validation
=====
>>> FFNN Base Model Validation Macro F1-Score: 0.7287

Training FFNN Depth 1 (2 Hidden Layer)...
=====
FFNN Depth 1 (2 Hidden Layer) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7235

Training FFNN Depth 2 (4 Hidden Layer)...
=====
FFNN Depth 2 (4 Hidden Layer) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7348

Training FFNN Depth 3 (5 Hidden Layer)...
=====
FFNN Depth 3 (5 Hidden Layer) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7353

=====
FFNN Base Model (8 Neuron) - Data Validation
=====
>>> FFNN Base Model Validation Macro F1-Score: 0.7287

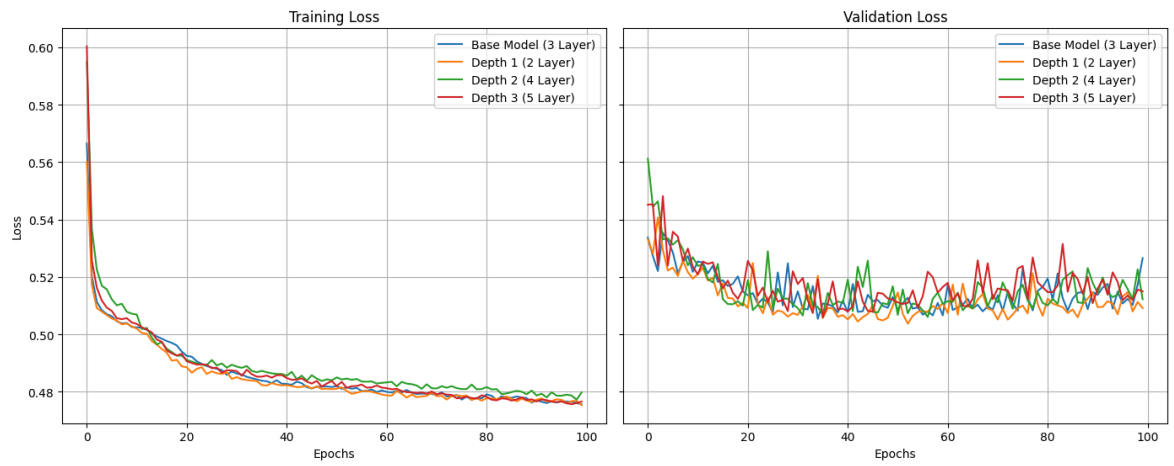
Training FFNN Width 1 (4 Neuron)...
=====
FFNN Width 1 (4 Neuron) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7349

Training FFNN Width 2 (16 Neuron)...
=====
FFNN Width 2 (16 Neuron) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7273

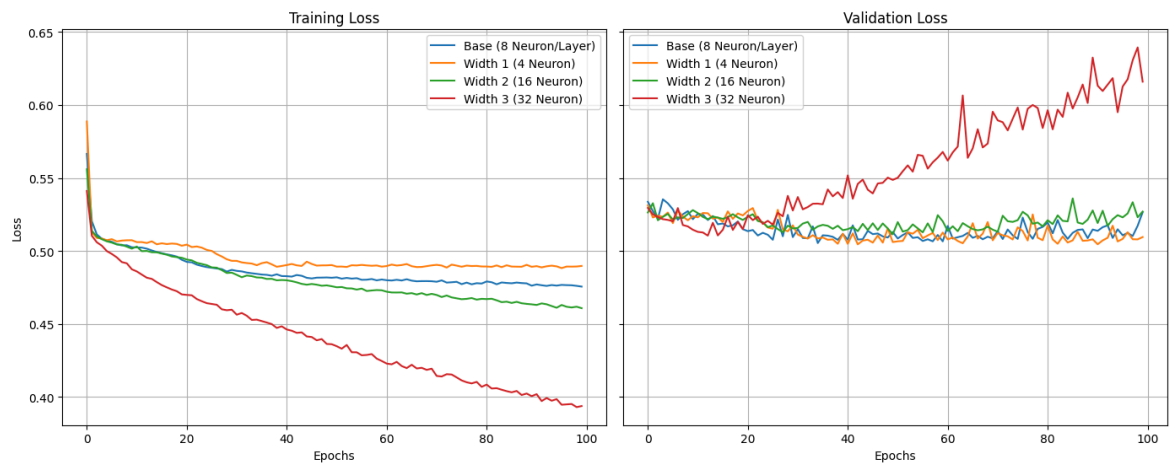
Training FFNN Width 3 (32 Neuron)...
=====
FFNN Width 3 (32 Neuron) - Data Validation
=====
>>> Validation Macro F1-Score: 0.6856
=====
```

Grafik Training Loss dan Validation Loss

Pengaruh Depth terhadap Model Loss



Pengaruh Width terhadap Model Loss



2.2.2. Pengaruh fungsi aktivasi

- Base arsitektur 3 hidden layer dengan fungsi aktivasi **ReLU**, output dengan fungsi aktivasi **sigmoid**
- **Layer pertama** sebagai layer pengujian
- Variasi fungsi aktivasi: Linear, Sigmoid, Hyperbolic Tangent, Binary Step, Softsign, Leaky ReLU

Fungsi aktivasi dengan nilai terbaik adalah Leaky ReLU dan sigmoid. Berbagai jenis fungsi aktivasi mempengaruhi bagaimana hasil perhitungan sigma diproses serta tingkat nilai lossnya.

Hasil Akurasi Prediksi

```
=====
FFNN Base Model (ReLU) - Data Validation
=====
>>> FFNN Base Model Validation Macro F1-Score: 0.7287

Training FFNN Variasi Aktivasi 1 (Linear)...

=====
FFNN Variasi Aktivasi 1 (Linear) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7288
```

```
Training FFNN Variasi Aktivasi 2 (Sigmoid)...

=====
FFNN Variasi Aktivasi 2 (Sigmoid) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7294

Training FFNN Variasi Aktivasi 3 (Tanh)...

=====
FFNN Variasi Aktivasi 3 (Tanh) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7237

Training FFNN Variasi Aktivasi 4 (Binary Step)...

=====
FFNN Variasi Aktivasi 4 (Binary Step) - Data Validation
=====
>>> Validation Macro F1-Score: 0.6697

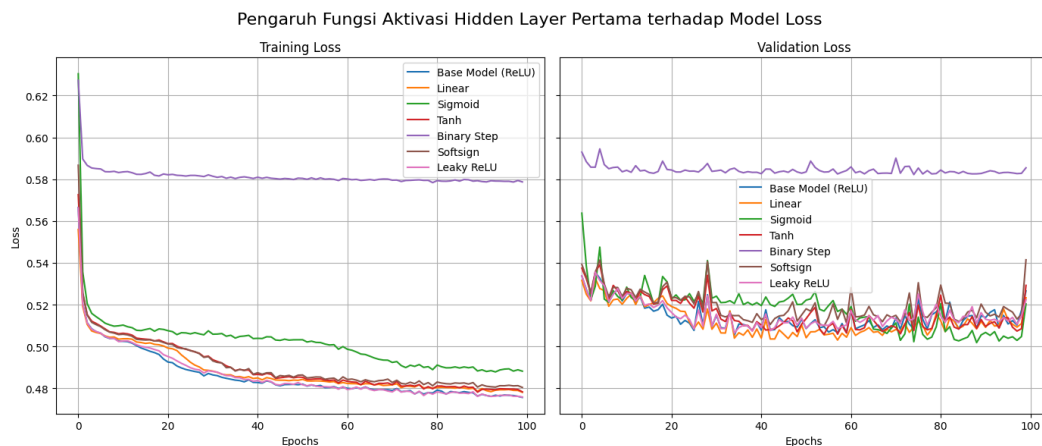
Training FFNN Variasi Aktivasi 5 (Softsign)...

=====
FFNN Variasi Aktivasi 5 (Softsign) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7202

Training FFNN Variasi Aktivasi 6 (Leaky ReLU)...

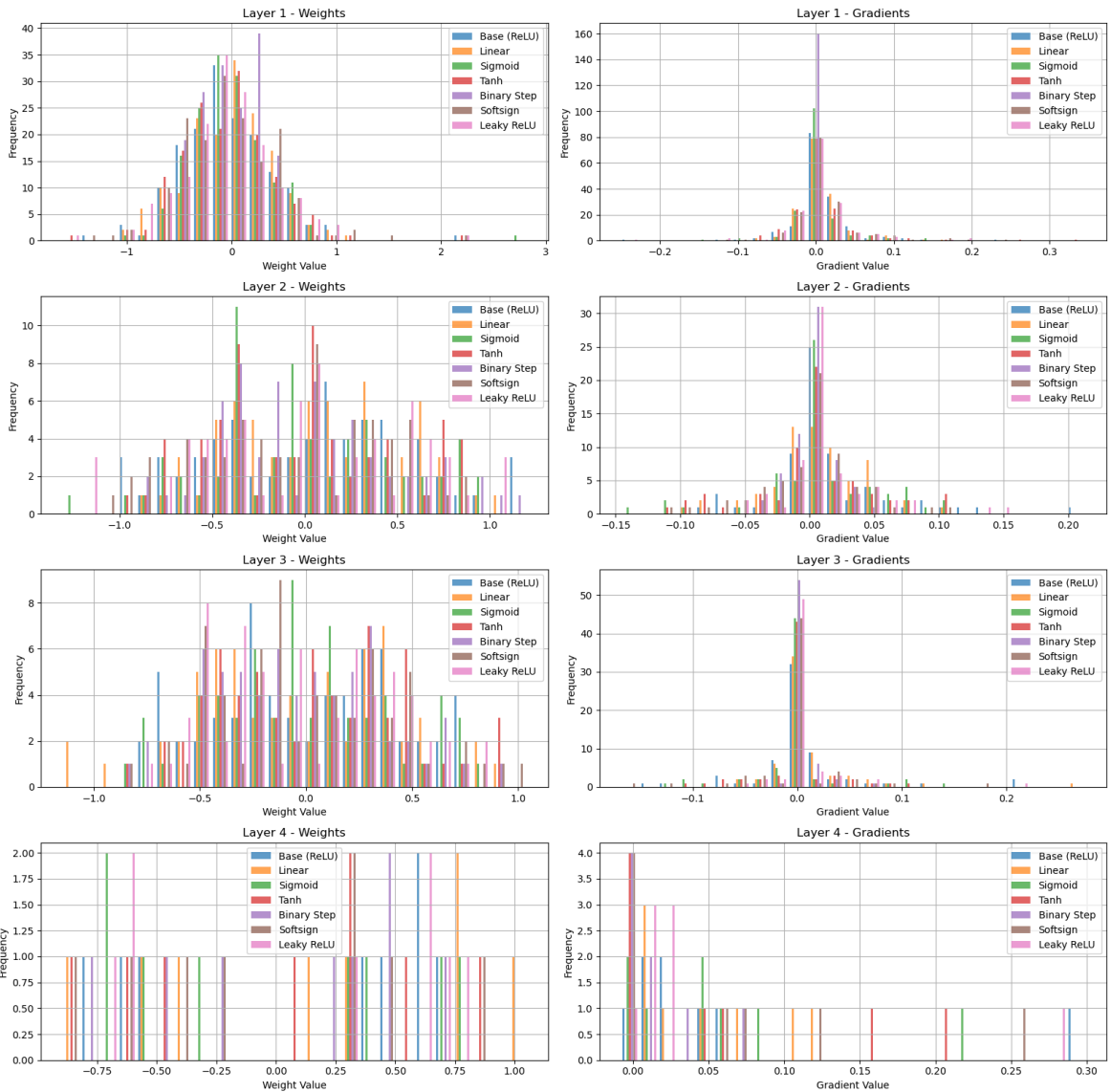
=====
FFNN Variasi Aktivasi 6 (Leaky ReLU) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7320
```

Grafik Training Loss dan Validation Loss



Distribusi Bobot dan Gradien Bobot

Perbandingan Distribusi Bobot dan Gradien - Variasi Fungsi Aktivasi



2.2.3. Pengaruh learning rate

- Base model menggunakan learning rate 0.1
- Variasi learning rate: 0.01, 0.05, 0.2

Learning rate yang paling tinggi terjadi pada nilai 0.01. Hubungan dari besarnya learning rate tidak bersifat linear karena tergantung pada jenis dataset. Dapat diinferensikan dari model dan data ini bahwa langkah kecil untuk perubahan bobot lebih dipilih daripada langkah besar. Pada grafik persebaran, dapat dilihat juga bahwa semakin besar learning rate, semakin besar distribusi weight valuenya.

Hasil Akurasi Prediksi

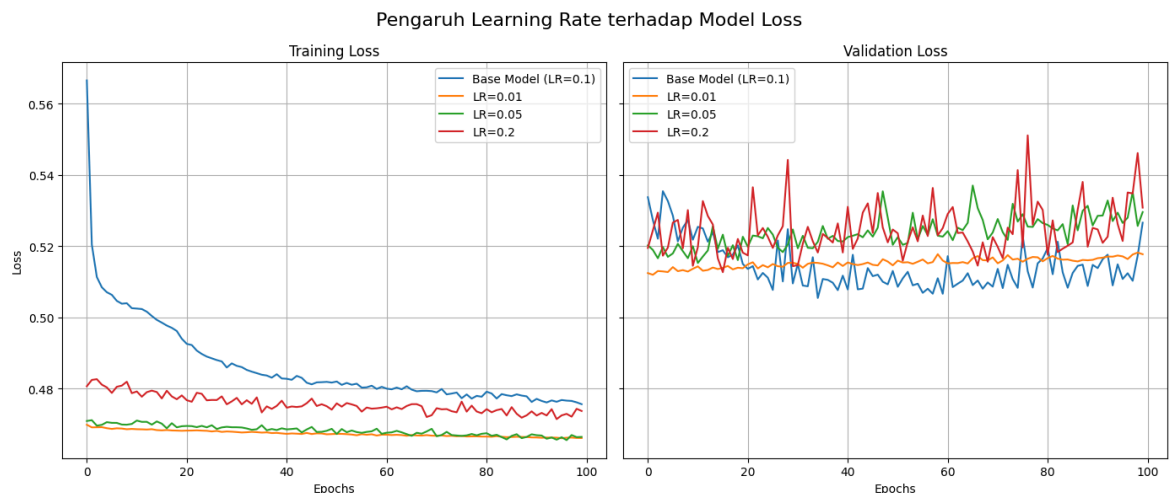
```
=====
FFNN Base Model (0.1) - Data Validation
=====
>>> FFNN Base Model Validation Macro F1-Score: 0.7287

Training FFNN Variasi Learning Rate 1 (0.01)...
=====
FFNN Variasi Learning Rate 1 (0.01) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7337

Training FFNN Variasi Learning Rate 2 (0.05)...
=====
FFNN Variasi Learning Rate 2 (0.05) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7254

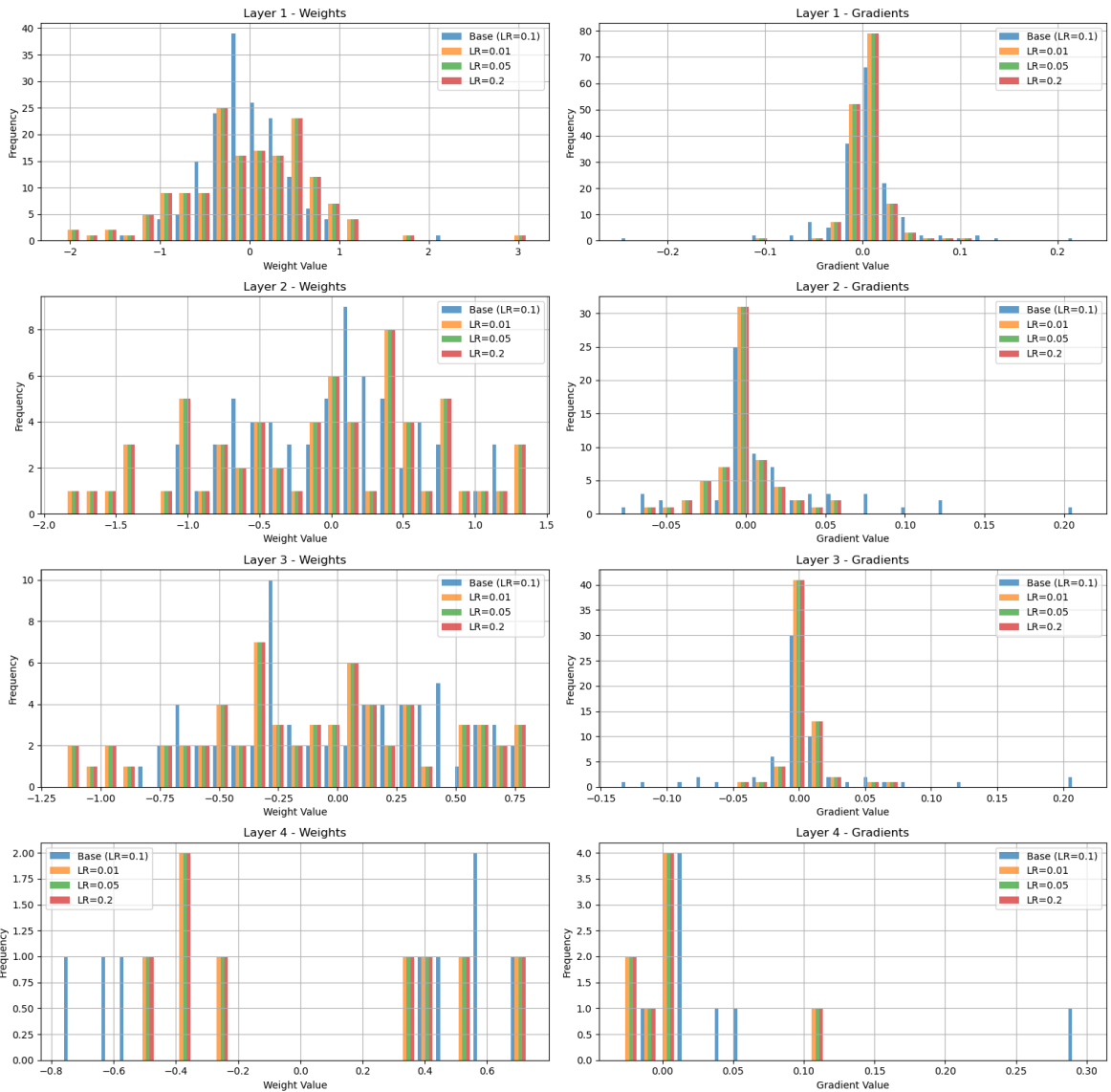
Training FFNN Variasi Learning Rate 3 (0.2)...
=====
FFNN Variasi Learning Rate 3 (0.2) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7250
```

Grafik Training Loss dan Validation Loss



Distribusi Bobot dan Gradien Bobot

Perbandingan Distribusi Bobot dan Gradien - Variasi Learning Rate



2.2.4. Pengaruh inisialisasi bobot

- Base model menggunakan inisialisasi bobot uniform
- Variasi inisialisasi bobot: zero, random normal, xavier, he

Inisialisasi bobot dengan Xavier memiliki nilai prediksi tertinggi. Perbedaan inisialisasi bobot berpengaruh terhadap besar loss per epoch. Untuk inisialisasi bobot zero, terjadi fenomena vanishing gradient yang menyebabkan training loss konvergen ke suatu nilai.

Hasil Akurasi Prediksi

```
=====
FFNN Base Model (Uniform) - Data Validation
=====
>>> FFNN Base Model Validation Macro F1-Score: 0.7287

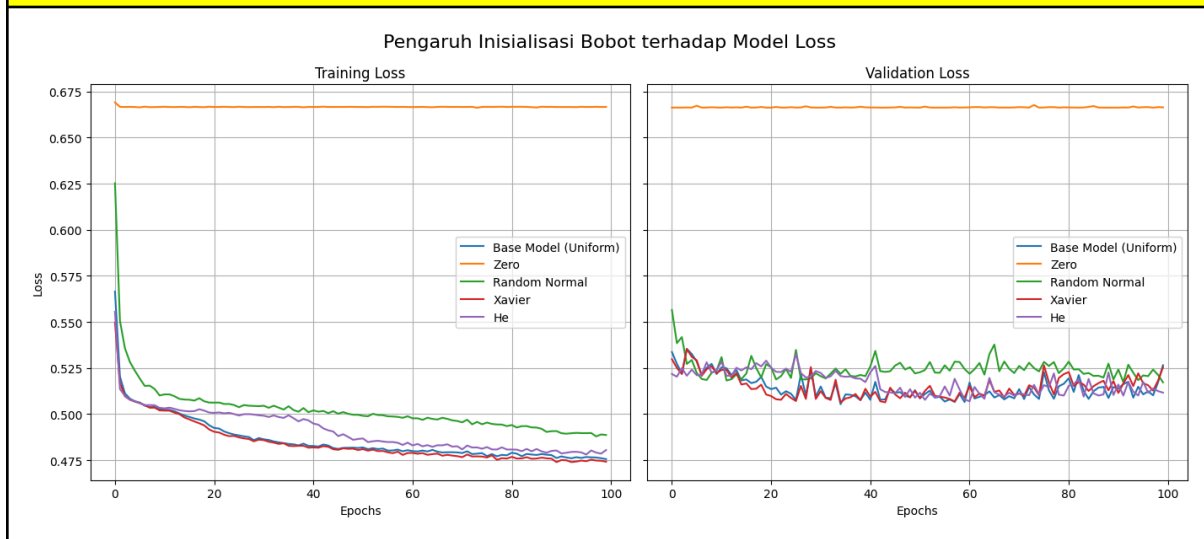
Training FFNN Variasi Inisialisasi Bobot 1 (Zero)...
=====
FFNN Variasi Inisialisasi Bobot 1 (Zero) - Data Validation
=====
>>> Validation Macro F1-Score: 0.3810

Training FFNN Variasi Inisialisasi Bobot 2 (Random Normal)...
=====
FFNN Variasi Inisialisasi Bobot 2 (Random Normal) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7283

Training FFNN Variasi Inisialisasi Bobot 3 (Xavier)...
=====
FFNN Variasi Inisialisasi Bobot 3 (Xavier) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7373

Training FFNN Variasi Inisialisasi Bobot 4 (He)...
=====
FFNN Variasi Inisialisasi Bobot 4 (He) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7307
```

Grafik Training Loss dan Validation Loss



2.2.5. Pengaruh regularisasi

- Regularisasi L1 dan L2 menggunakan $\lambda = 0.01$
- Variasi pengujian: tanpa regularisasi (base model), L1 regularisasi, L2 regularisasi

Pada dataset dan model ini, model tanpa regularisasi justru lebih baik. Hal ini dapat ditafsirkan sebagai perubahan bobot yang terlalu drastis menyebabkan model ini kurang efektif, walaupun secara teori dapat meningkatkan nilai prediksi (dengan mengecilkan bobot yang lemah atau mengurangi bobot keseluruhan). Pada grafik distribusi, dapat dilihat bahwa model dengan L1 dan L2 memiliki persebaran bobot yang terpusat di nilai 0, berbeda dengan model tanpa regularisasi yang lebih tersebar.

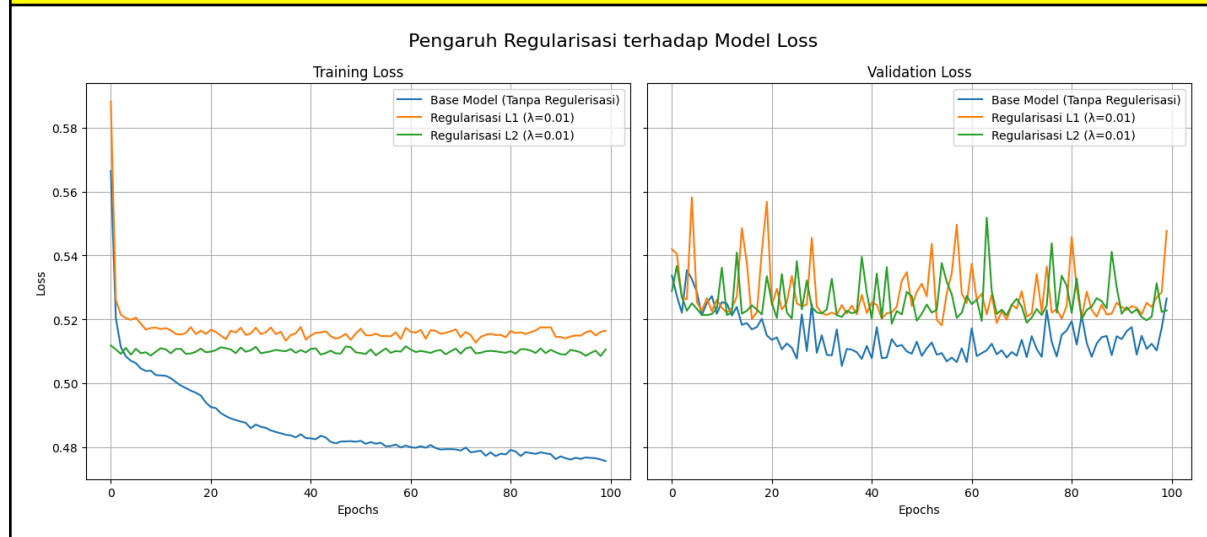
Hasil Akurasi Prediksi

```
=====
FFNN Base Model (Tanpa Regularisasi) - Data Validation
=====
>>> FFNN Base Model Validation Macro F1-Score: 0.7287

Training FFNN Variasi Regularisasi L1...
=====
FFNN Variasi Regularisasi L1 - Data Validation
=====
>>> Validation Macro F1-Score: 0.7217

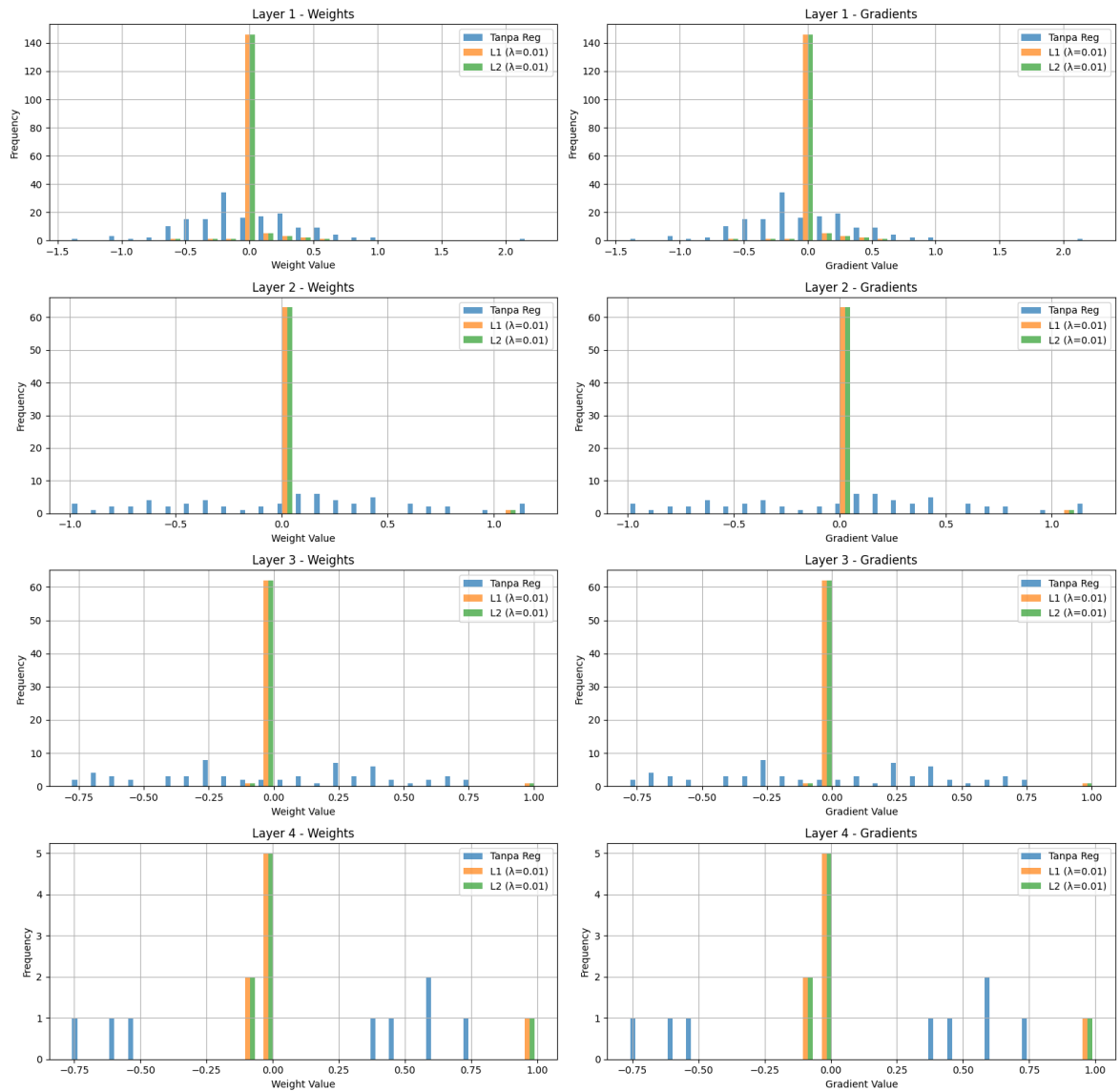
Training FFNN Variasi Regularisasi L2...
=====
FFNN Variasi Regularisasi L2 - Data Validation
=====
>>> Validation Macro F1-Score: 0.7240
```

Grafik Training Loss dan Validation Loss



Distribusi Bobot dan Gradien Bobot

Perbandingan Distribusi Bobot dan Gradien - Variasi Regularisasi



2.2.6. Perbandingan dengan library sklearn

- Perbandingan hasil prediksi dengan library sklearn MLP
- Menggunakan hyperparameter yang sama

Hasil model yang dibangun menggunakan parameter yang sama lebih baik daripada MLP model dari sklearn.

Hasil Akurasi Prediksi
<pre>===== FFNN Base Model - Data Validation ===== >>> FFNN Base Model Validation Macro F1-Score: 0.7287 Training sklearn MLP Model... ===== sklearn MLP Model - Data Validation ===== >>> MLPClassifier Validation Macro F1-Score: 0.7026</pre>

2.2.7. Perbandingan dengan RMS Normalization

- Perbandingan dengan RMS Normalization
- Hasil prediksi berbeda sedikit

Kurva loss lebih cepat pada RMS dibanding tanpa normalisasi. Hal ini disebabkan oleh RMS yang menyeragamkan nilai terlebih dahulu sehingga nilai menuju loss dapat dicapai oleh gradient descent dengan mudah. Pada gradien bobot, grafik gradien tanpa normalisasi kebanyakan berpusat pada 0.0, berbeda dengan RMS yang lebih tersebar. Hal ini disebabkan karena RMS menjaga agar nilai aktivasi tidak terlalu besar maupun kecil sehingga tidak menyebabkan gradien meledak atau menghilang.

Hasil Akurasi Prediksi

```

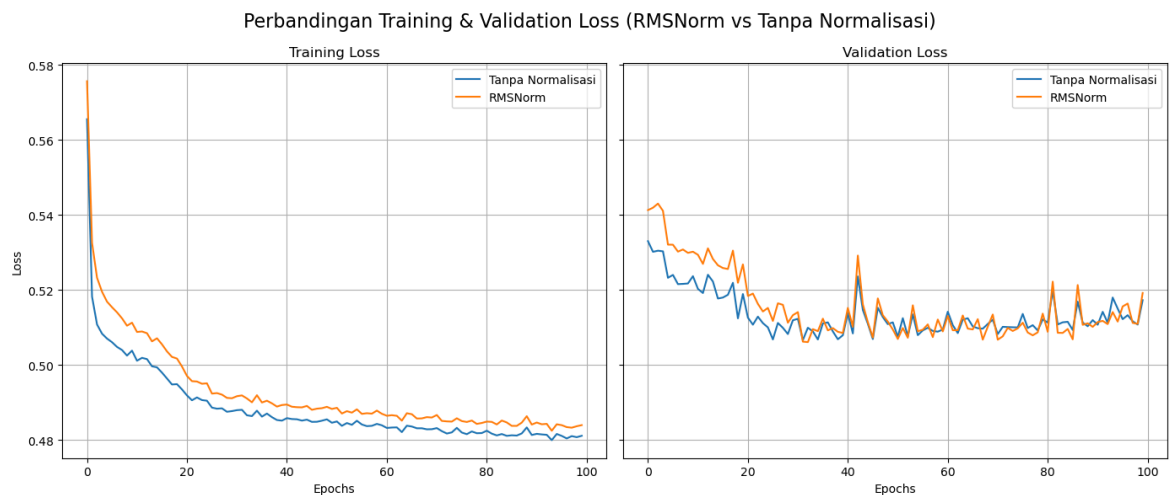
... Training Base Model (Tanpa Normalisasi)...
=====
Base Model (Tanpa Normalisasi) - Data Validation
=====
>>> Validation Macro F1-Score: 0.7193

Training Model dengan RMSNorm...
=====
Model dengan RMSNorm - Data Validation
=====
>>> Validation Macro F1-Score: 0.7196

```

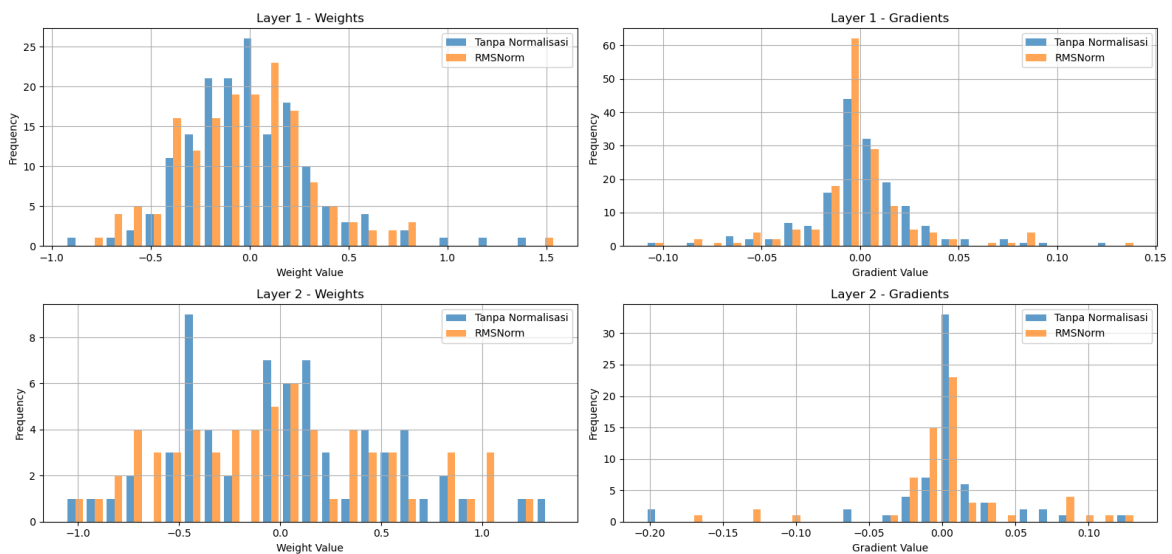
-

Grafik Training Loss dan Validation Loss



Distribusi Bobot dan Gradien Bobot

Distribusi Bobot dan Gradien: RMSNorm vs Base Model



3. Kesimpulan dan Saran

3.1. Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, model Feed Forward Neural Network (FFNN) berhasil diimplementasikan tanpa menggunakan library dan menghasilkan prediksi model yang lebih bagus dari model MLP dari library sklearn. Melalui tugas ini, kami mengeksplorasi pemodelan dasar arsitektur FFNN yang terdiri dari Layer dengan algoritma backpropagation. Kami juga mengeksplorasi pengujian untuk menentukan hubungan antara parameter-parameter terhadap hasil prediksi akhir model serta hubungannya terhadap grafik loss dan distribusi bobot. Parameter yang diujikan adalah width, depth, learning rate, fungsi aktivasi, inisialisasi bobot, regularisasi, dan penggunaan RMSNormalization. Hasil pengujian menunjukkan bahwa tiap pengaturan berpengaruh terhadap hasil akhir prediksi dan loss, namun harus dipertimbangkan bahwa penemuan dalam pengujian tidak menunjukkan hubungan yang berlaku untuk semua kasus lainnya, namun berlaku untuk pemodelan yang telah dibuat serta dataset yang digunakan.

3.2. Saran

Berdasarkan hasil eksperimen dan analisis yang telah dilakukan, berikut adalah beberapa saran untuk pengembangan model di masa mendatang:

- Melakukan pengujian berdasarkan parameter jumlah epoch
- Melakukan pengujian berdasarkan parameter fungsi loss
- Melakukan pengujian berdasarkan parameter batch size
- Melakukan pengujian menggunakan automatic differentiation untuk memudahkan perhitungan gradien
- Melakukan pengujian menggunakan metode lain selain gradient descent seperti adaptive moment estimation (adam)

Hal ini ditujukan untuk mengeksplorasi dan menemukan hubungan antara pengaturan parameter tersebut dengan model FFNN yang diimplementasikan agar menghasilkan model FFNN yang paling optimal.

4. Pembagian Tugas

Nama Anggota	Pembagian Tugas
Nicholas Andhika Lucas (13523014)	1. Membuat tahapan EDA dan Preprocessing 2. Membantu pembuatan model FFNN pada definisi loss functions dan seed 3. Melakukan pengujian dan visualisasi pengujian 4. Menambahkan bonus 3 fungsi aktivasi lainnya 5. Menyusun laporan bagian deskripsi persoalan, deskripsi kelas, bonus, pengujian, dan kesimpulan
Stefan Mattew Susanto (13523020)	1. Membuat kelas Activation dan Layer 2. Membuat model kelas FFNN 3. Membuat forward propagation 4. Menyusun laporan bagian deskripsi kelas, forward propagation
Kenneth Ricardo Chandra (13523022)	1. Membuat backward propagation 2. Membuat bonus He dan Xavier initialization 3. Membuat RMS normalization 4. Menyusun laporan bagian backward propagation, weight update dan bonus

5. Referensi

[Memahami Data Dengan Exploratory Data Analysis | by AC | Data Folks Indonesia | Medium](#)

[Histogram in matplotlib | PYTHON CHARTS](#)

[Activation function - Wikipedia](#)

[MLPClassifier — scikit-learn 1.8.0 documentation](#)

[Understanding L1 and L2 Regularization | Towards Data Science](#)

<https://www.geeksforgeeks.org/deep-learning/what-is-forward-propagation-in-neural-networks/>

<https://www.datacamp.com/tutorial/feed-forward-neural-networks-explained>

<https://medium.com/@fajri.hulvi/forward-dan-backward-propagation-fundamental-dari-belajar-neural-network-809d79dd9c96>

6. Lampiran Form Penggunaan AI

FORMAT DEKLARASI PENGGUNAAN AI SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM :
1. Nicholas Andhika Lucas/13523014
2. Stefan Matthew Susanto /13523020
3. Kenneth Ricardo Chandra/13523022
Fakultas/Sekolah : STEI-K
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 1 IF3270 Feedforward Neural Network

Menyatakan bahwa **kami menggunakan** AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	AI-assisted idea generation and structuring, AI-assisted workflow
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.



Nicholas Andhika Lucas
13523014



Stefan Matthew Susanto
13523020

Jakarta, 18 Maret 2026



Kenneth Ricardo Chandra
13523022